NAME:

Project 1: Drawings with Turtle

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

Learning Objectives:

1. Practice designing and implementing simple algorithms.
2. Demonstrate that you can use the programming concepts that we've covered so far: function calls, function definitions, variables and assignment, and counting loops
3. Practice reading the official Python documentation.
4. More practice defining your own functions and using them.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Drawing shapes

1.1 Square

1. Open the starter code `square.py`. You'll notice that a few variables are defined at the top of the file. Please use these variables in your code.
2. Add some code that moves the turtle to the (`x`,`y`) position without drawing a line.
3. Add some code that draws the square using size as the side length.
4. Make sure to save your program in a file called `square.py`.
5. Make sure that your code is fully commented.

1.2 Rectangle

1. Create a new code file called rectangle.py using a similar format to `square.py`. Meaning:
    a. Define `x, y` to indicate the starting position for the rectangle.
    b. Create: width and height to control the size of the rectangle.
    c. The entire rectangle should be visible in the turtle window without having to resize it, but beyond this restriction, feel free to place the rectangle wherever you like and to make it as big or small as you like.
2. Add code to move the turtle to position (`x`,`y`).
3. Add code to draw a rectangle with the specified width and height.
4. Save this code in a file called `rectangle.py` and make sure to comment the code.
    - Hint: It may be very helpful to copy and paste code from `square.py` to reduce the amount of typing you have to do.

1.3 Circle

1. Follow the same general steps for the rectangle, except this time you'll be creating a circle.
    a. File name: `circle.py`.
    b. Variables to create: `x, y, radius`.

1.4 Triangle

1. Follow the same general steps as for the rectangle, but this time we're making a triangle.
    a. File name: `triangle.py`.
    b. Variables to create: `x, y, size`.
2. This program should draw an equilateral triangle.
3. The size variable should indicate the length of one side.

2, Functions for shapes

1. Open the starter file `turtle_shapes.py`. This file contains the beginnings of some function definitions. (We'll be discussing function definitions more this week)
2. Copy your code for drawing a square from `square.py` into the `square(x, y, size)` function definition. There are comments explaining how to do this in the starter file.
    - Caution: Do not copy the variable definitions at the top of the square.py file. They will not be needed if you designed your square.py code correctly.
3. You should notice that the `square(x, y, size)` function is called at the bottom of the starter file. Once you've completed the previous step, running the program should produce a drawing of a square in the bottom right quadrant of the turtle window.
4. Repeat step 2 for the other shapes (rectangle, circle, triangle).
    - Caution: Don't change the order of the parameters in the function definitions.
5. Add function calls for each of the shapes at the bottom of the file to test your program and prove that the functions work as expected.
6. Please note that you should copy over comments in addition to the code itself.

3. Draw a picture

1. Create a new file and copy the function definitions from Part 2 into it. Save this new file as `my_picture.py`.
2. Write a program that uses these functions to draw a picture.
3. Picture requirements:
    a. Use at least two line colors and at least two fill colors.
    - Hint: it may be useful to refer back to your in-class assignments. If you're still having trouble, take a look at the `turtle` library section of the Python official documentation (https://docs.python.org/3/library/turtle.html)… or stop by my office hours.
    b. Your picture should depict a recognizable object or scene. For example, something like the picture bloew (though it does not need to be this involved)

c. You may add additional functions to help draw the picture (e.g. functions to draw filled-in versions of each shape), but you should not change the functions that you defined for Part 2 of the assignment.
d. Your program should use each function from Part 2 at least once, but you can also use other built-in turtle functions.
e. Your program should use loops when appropriate to keep your code DRY (Don't Repeat Yourself).

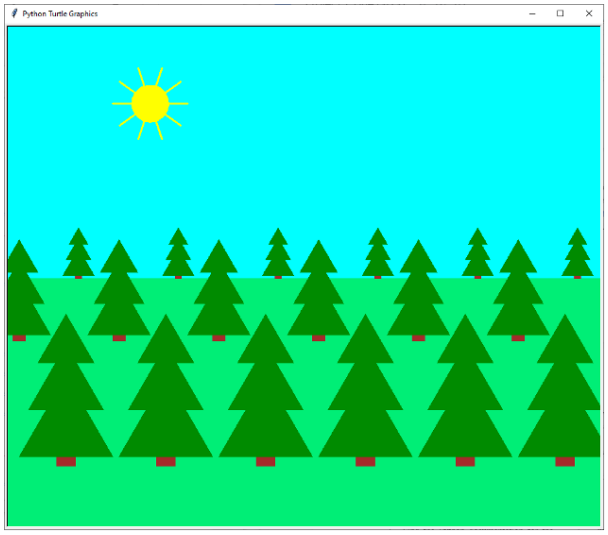

Figure 1: Turtle Drawing

4. Computer Generated art

1. Find the Python documentation for the `random` module. (Hint: Use the link to the "library reference" on the Resources page on the course website)
2. The function `randrange` from the `random` module produces a random integer. Test it out by writing a program that generates five random numbers between 2 and 9 (inclusive) and prints them out. Save your program in a file called `five_random_numbers.py`. Hint: remember to keep your code DRY.
3. Open the starter file `computer_art.py`.
4. This file already contains definitions for two functions. In particular, it contains a function called `random_shape` that randomly picks a shape (square, rectangle, circle, or triangle) and draws it. The function takes three parameters:

    a. `x` - the x location
    b. `y` - the y location
    c. `size` - the size

5. Copy and paste your function definitions from Part 2 into this file.
6. Add some code to draw 30 randomly picked shapes at random locations on the screen. For example, your output could look something like this:

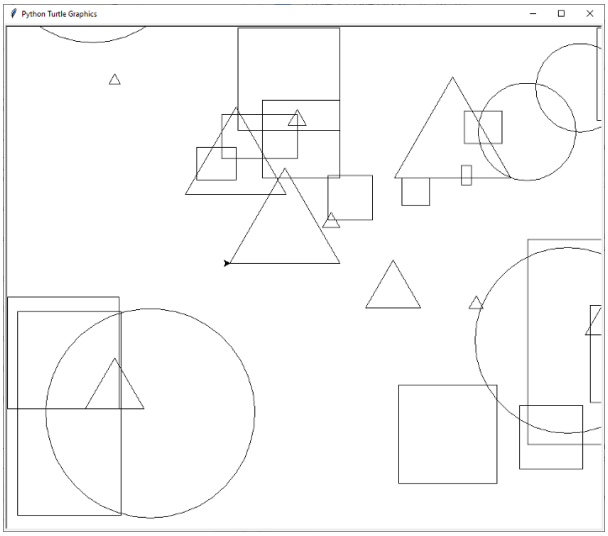

Figure 2: Computer Art

7. (Optional): Add some code to randomly change the pensize, pencolor, and filcolor.

5. What to submit

You will need to submit the following files:

1. `square.py`
2. `rectangle.py`
3. `circle.py`
4. `triangle.py`
5. `turtle_shapes.py`
6. `my_picture.py`
7. `five_random_numbers.py`
8. `computer_art.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:
    a. A header comment, which includes your name, the date, the assignment and a brief description of the program.
    b. Comments within the body of the code to further clarify how the program works.
2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code?
    - Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?
    a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting.

Once you have checked these things, submit your code to:

"Project 1: Drawings with Turtle" on Gradescope.

6. Grading guidelines

- **Demo specifics to come** … generally speaking, focus on the below elements
- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.